

Recuit simulé pour le TSPTW

Implémentation, calibration et analyse expérimentale

Charles Gaye

Raphaël Fromager

Janvier 2026

Cours : Complément Parcours Recherche — Info 2

Contents

1	Introduction	2
2	Modélisation et algorithme	2
2.1	Représentation des solutions	2
2.2	Fonction de score	2
2.3	Voisinages	2
2.4	Recuit simulé et schéma de refroidissement	3
3	Implémentation	3
4	Méthodologie expérimentale	4
5	Résultats	4
5.1	Calibration des températures	4
5.2	Choix du voisinage	5
5.3	Pénalités	5
5.4	Résultats par instance	6
6	Discussion et limites	7
6.1	Erreur d'implémentation sur la pénalité d'avance	7
6.2	Coefficient sur les violations	8
6.3	Limites de la fonction de voisinage	9
6.4	Écart à l'optimum sur l'instance de concours	9
7	Conclusion	9

1 Introduction

Ce rapport présente la démarche et les résultats du projet de métaheuristique consacré au problème du voyageur de commerce avec contraintes temporelles (TSPTW, *Travelling Salesman Problem with Time Windows*). On se donne un ensemble de villes munies de coordonnées planes et d’une fenêtre de visite $[w_{\text{start}}, w_{\text{end}}]$, et on cherche un ordre de visite, fixant la première ville et revenant au point de départ, qui minimise la distance totale parcourue tout en respectant les fenêtres.

Le projet portait sur quatre instances : trois fournies à des fins de mise au point (de tailles respectives ~ 20 , ~ 50 et ~ 200 villes), et une instance de concours de taille intermédiaire (~ 100 villes). Notre objectif était d’implémenter et de calibrer un algorithme de recuit simulé, méthode jugée raisonnable au regard de notre niveau de programmation et bien adaptée aux problèmes combinatoires de cette taille. L’algorithme de colonie de fourmis avait été envisagé comme alternative, mais sa mise au point était estimée trop coûteuse en temps de débogage pour un projet pédagogique.

Ce rapport est volontairement descriptif : il documente les choix faits, ce qu’ils ont permis d’obtenir, et les limites qui ont été identifiées au cours du travail (y compris une erreur d’implémentation détectée a posteriori). Il ne vise pas à présenter une méthode optimisée, et les résultats numériques rapportés sont strictement ceux qui ont été obtenus pendant le projet.

2 Modélisation et algorithme

2.1 Représentation des solutions

Une solution à une instance de N villes est une permutation des entiers $1, \dots, N$ qui commence par la ville imposée comme dépôt (numérotée 1 par convention). L’espace des solutions est ainsi l’ensemble des permutations de $\{2, \dots, N\}$, augmenté du préfixe 1, soit $(N - 1)!$ éléments. La validité d’une solution dépend du respect des fenêtres temporelles : on calcule, le long du tour, la date courante d’arrivée à chaque ville en autorisant l’attente si l’on arrive avant l’ouverture de la fenêtre, et on compte une violation à chaque fois que la date d’arrivée dépasse la fermeture.

2.2 Fonction de score

Faute de pouvoir restreindre rigoureusement la recherche aux solutions admissibles, on travaille avec une fonction de score pénalisée qui prend la forme :

$$\text{score}(s) = \text{distance}(s) + p_v n_{\text{viol}}(s) + p_r t_{\text{retard}}(s) + p_a t_{\text{avance}}(s), \quad (1)$$

où n_{viol} compte les villes visitées hors fenêtre, t_{retard} est le retard cumulé en unités de temps et t_{avance} le temps d’attente cumulé. Les coefficients p_v , p_r et p_a sont des paramètres à calibrer. On notera que le terme d’avance, par construction, ne pénalise pas la qualité « voyageur » d’une solution mais traduit plutôt une volonté de regrouper les visites — ce point sera rediscuté en section 6.

2.3 Voisinages

La fonction de voisinage produit, à partir d’une solution courante, une nouvelle solution candidate. Un premier essai naïf consistait à appliquer une transposition de deux villes (sans toucher à la ville 1). Cette opération s’est révélée trop locale : sur l’instance la plus simple, l’algorithme convergeait fréquemment vers des solutions où il satisfaisait les fenêtres temporelles en ne réagénçant qu’un petit nombre de villes, sans réellement explorer l’espace.

Quatre opérateurs ont été retenus, qui agissent sur des blocs contigus :

- `large_swap` : échange de deux blocs contigus disjoints de même taille.

- **block_reverse** : inversion de l'ordre dans un bloc.
- **scramble** : permutation uniforme à l'intérieur d'une sous-séquence.
- **insert_range** : extraction d'un bloc et réinsertion à une autre position.

À chaque appel, l'opérateur appliqué est tiré selon un vecteur de probabilité $p = (p_1, p_2, p_3, p_4)$. En complément, lorsque la solution courante présente au moins une violation de fenêtre, un opérateur de réparation est tiré 30 % du temps : il sélectionne une ville en retard et la réinsère plus tôt dans le tour.

2.4 Recuit simulé et schéma de refroidissement

L'algorithme suit le schéma classique : à chaque itération, on tire un voisin s' de la solution courante s , et on l'accepte avec probabilité 1 si $\Delta = \text{score}(s') - \text{score}(s) \leq 0$, sinon avec probabilité $\exp(-\Delta/T)$ (distribution de Gibbs). La température T décroît de manière géométrique :

$$T \leftarrow \lambda T, \quad \lambda = \left(\frac{T_{\text{end}}}{T_{\text{start}}} \right)^{1/N_{\text{iter}}}, \quad (2)$$

de sorte que T atteint exactement T_{end} après N_{iter} itérations.

```

Function simulated_annealing( $T_{\text{init}}, T_{\text{end}}$ )
    current_best  $\leftarrow$  rand_solution()
     $T \leftarrow T_{\text{init}}$ 
    while  $T < T_{\text{end}}$  do
         $s' \leftarrow$  rand_neighbor( $s$ )
        if  $E(s') < E(s)$  then
            | proba  $\leftarrow$  1
        else
            | proba  $\leftarrow \exp(-\frac{E(s')-E(s)}{T})$ 
        if rand() < proba then  $s \leftarrow s'$ ;
         $T \leftarrow$  update( $T$ )
    return s

```

Figure 1: Schéma du recuit simulé.

3 Implémentation

Le code est écrit en Python 3 et organisé en modules :

- **io_utils** — lecture des instances, calcul et mise en cache de la matrice de distances avec fermeture pour l'inégalité triangulaire, lecture/écriture des fichiers solution.
- **scoring** — fonction de score pénalisée.
- **neighborhoods** — opérateurs de voisinage et heuristique de réparation.
- **sa** — boucle de recuit simulé, paramétrée par une dataclass **SACConfig**.
- **experiments** — moyennage de plusieurs essais et balayages de paramètres.
- **plotting** — tracés de progression et de comparaison avec intervalle de confiance.

Un point d'entrée `main_new.py` expose trois modes d'exécution (un seul recuit, balayage de probabilités de voisinage, balayage de pénalités) avec une graine aléatoire optionnelle pour la reproductibilité.

Tâche initialement sous-estimée : le calcul de la matrice de distances avec fermeture triangulaire est en $\mathcal{O}(N^3)$. Sur l'instance de concours (~ 100 villes), il est exécuté une seule fois par instance puis mis en cache au format `.npy`, ce qui rend le coût négligeable pour les expériences répétées.

4 Méthodologie expérimentale

Pour comparer deux paramétrages, exécuter un seul recuit ne suffit pas : la variance entre essais peut masquer ou simuler des écarts de performance. On a mis en place une procédure de moyennage qui consiste à exécuter, pour chaque configuration, n_{runs} recuits indépendants et à empiler les courbes du meilleur score atteint au fil des itérations.

Le score affiché dans les comparaisons est calculé sous une métrique de référence *indépendante* du paramétrage testé (pénalités fixées à $p_v = 500$, $p_r = 25$, $p_a = 3$). Cela permet de comparer entre eux des recuits qui auraient utilisé des pénalités différentes pour piloter leur trajectoire, sans que la métrique d'évaluation soit elle-même sensible au paramétrage.

On reporte la moyenne et un intervalle de confiance à 95 % autour de cette moyenne, calculé en supposant les essais indépendants et identiquement distribués :

$$\text{IC} = \bar{X} \pm 1.96 \frac{\sigma}{\sqrt{n_{\text{runs}}}}. \quad (3)$$

Cet IC borne la moyenne d'une configuration à un instant donné ; il ne quantifie pas la variabilité d'un essai isolé. Cette nuance jouera un rôle dans la discussion (section 6).

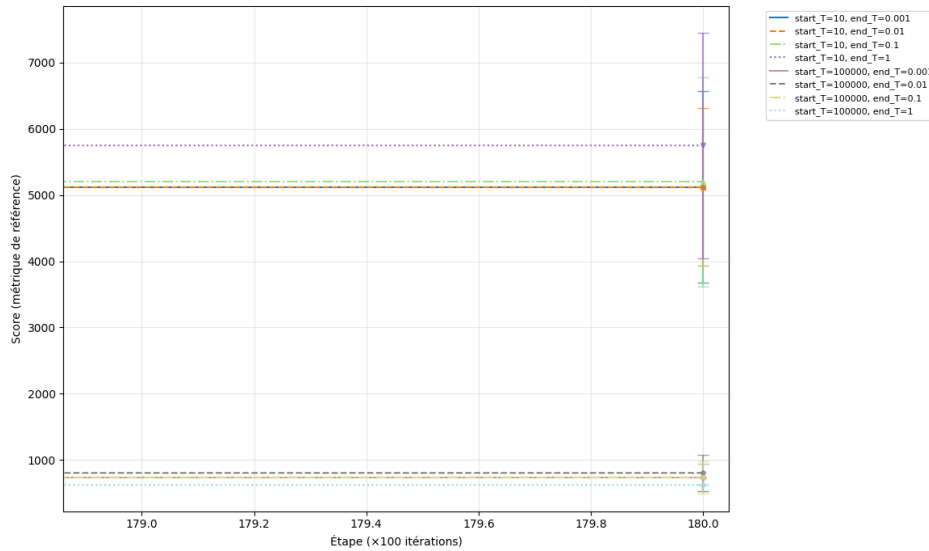


Figure 2: Premier balayage de la température initiale.

5 Résultats

5.1 Calibration des températures

Les premières valeurs essayées ($T_{\text{start}} \approx 100$, $T_{\text{end}} \approx 0.01$, environ 5 000 itérations) ne permettaient pas de trouver de solution admissible. Le diagnostic, posé après une discussion en cours, a été

que ces paramètres faisaient décroître la température trop vite : la probabilité d’accepter un voisin de moins bonne qualité devenait rapidement négligeable, et l’algorithme se comportait en pratique comme une descente locale dès les premières itérations.

En relevant la température initiale dans la plage $[10^4, 10^5]$ et en augmentant le nombre d’itérations à environ 50 000, l’algorithme a commencé à produire des solutions sans retard sur l’instance 1. Un balayage exponentiel de T_{start} sur $[10^3, 10^6]$, affiné progressivement, a conduit à retenir $T_{\text{start}} = 5\,000$ comme valeur usuelle. La température finale s’est révélée plus difficile à fixer : nous avons constaté empiriquement qu’une valeur plus faible était nécessaire pour les instances comportant davantage de villes (≈ 1 pour l’instance 1, $\approx 0.05\text{--}0.5$ pour les instances 2 et 3, ≈ 0.1 retenu pour l’instance de concours, qui est de taille intermédiaire). Cette dépendance n’a pas été quantifiée par une expérience dédiée.

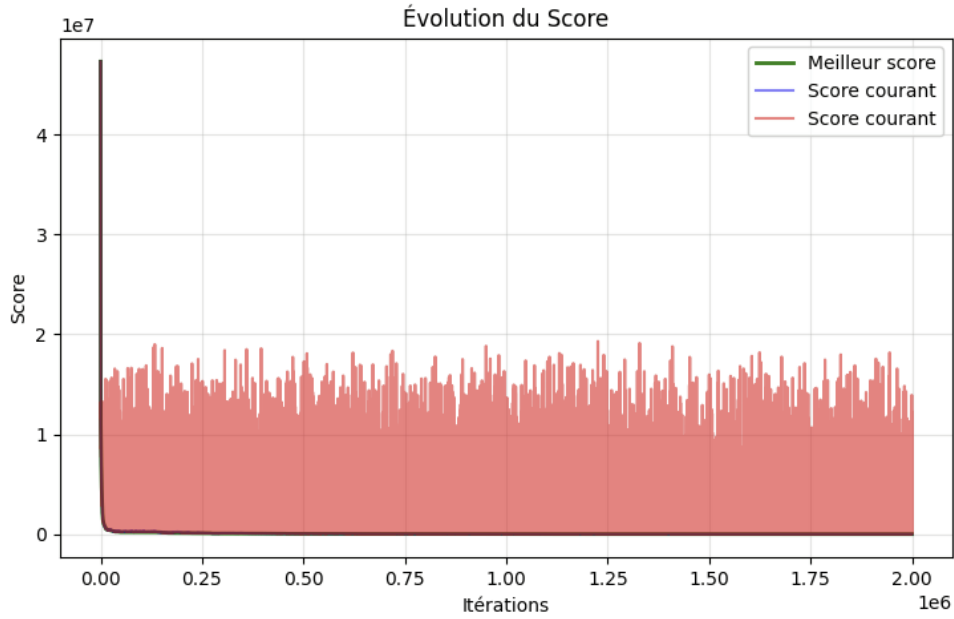


Figure 3: Évolution typique du score au cours d’un recuit (instance 3).

5.2 Choix du voisinage

L’effet du vecteur p de probabilités de voisinage a été étudié en plusieurs étapes. Une première comparaison entre les vecteurs unitaires e_1, e_2, e_3, e_4 (un seul opérateur actif) a fait ressortir **scramble** (i.e. e_3) comme le plus performant. Des combinaisons à dominante e_3 (par exemple $p = (0, 0.1, 0.75, 0.15)$) ont donné des résultats comparables au e_3 pur, sans qu’on ait pu mettre en évidence de combinaison strictement meilleure : tous les vecteurs testés à dominante **scramble** produisaient des courbes dont les intervalles de confiance se chevauchaient.

Les vecteurs retenus pour la suite étaient donc $p = (0, 0, 1, 0)$ et $p = (0, 0.1, 0.8, 0.1)$, ainsi que des variantes proches.

5.3 Pénalités

La fonction de score retenue après calibration est :

$$\text{score} = \text{distance} + 350 n_{\text{viol}} + 30 t_{\text{retard}} + 10 t_{\text{avance}}. \quad (4)$$

Le coefficient 350 sur les violations a été abaissé par rapport à la valeur initiale (10 000) pour ne pas écraser les autres termes : avec une pénalité trop forte, l’algorithme pouvait satisfaire toutes les fenêtres en restant sur des solutions de mauvaise qualité au sens de la distance.

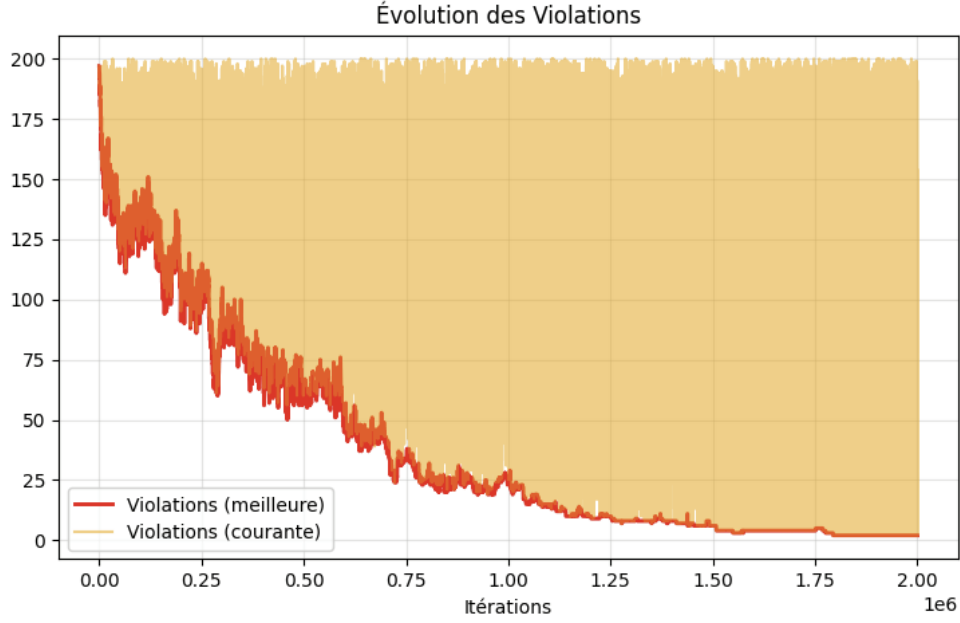
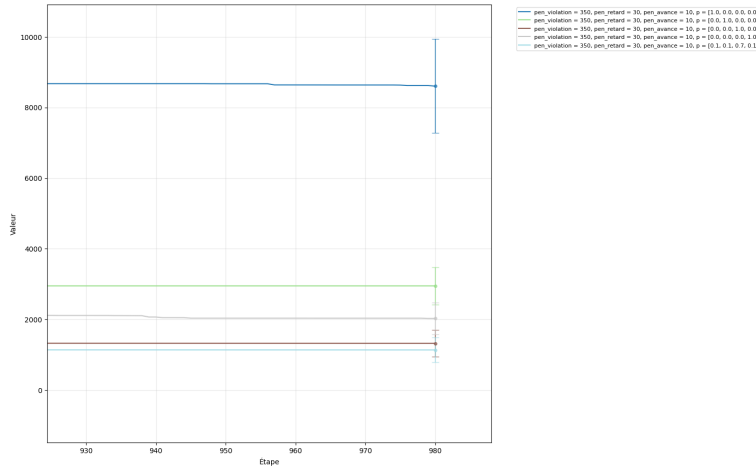


Figure 4: Évolution typique du nombre de violations (instance 3).

Figure 5: Comparaison des vecteurs unitaires et d'un vecteur à dominante **scramble**.

Un effet a été observé sur le coefficient d'avance : le groupe avec $p_a = 10$ obtenait, sur l'instance de concours, une moyenne finale plus basse que le groupe avec $p_a = 5$, avec des intervalles de confiance qui se chevauchaient peu (cf. figure suivante). Ce point est rediscuté en section 6.1.

5.4 Résultats par instance

Avec le paramétrage final ($T_{\text{start}} = 5\,000$, $T_{\text{end}} \approx 0.1$, de 10^5 à 2×10^6 itérations selon l'instance, p à dominante **scramble**, pénalités (350, 30, 10)) :

- **Instance 1** (~ 20 villes) : la solution optimale donnée avec le sujet est retrouvée systématiquement dès le premier essai pour un nombre d'itérations suffisant.
- **Instance 2** (~ 50 villes) : la solution optimale est retrouvée.
- **Instance 3** (~ 200 villes) : aucune solution admissible n'a été trouvée. Les meilleurs essais

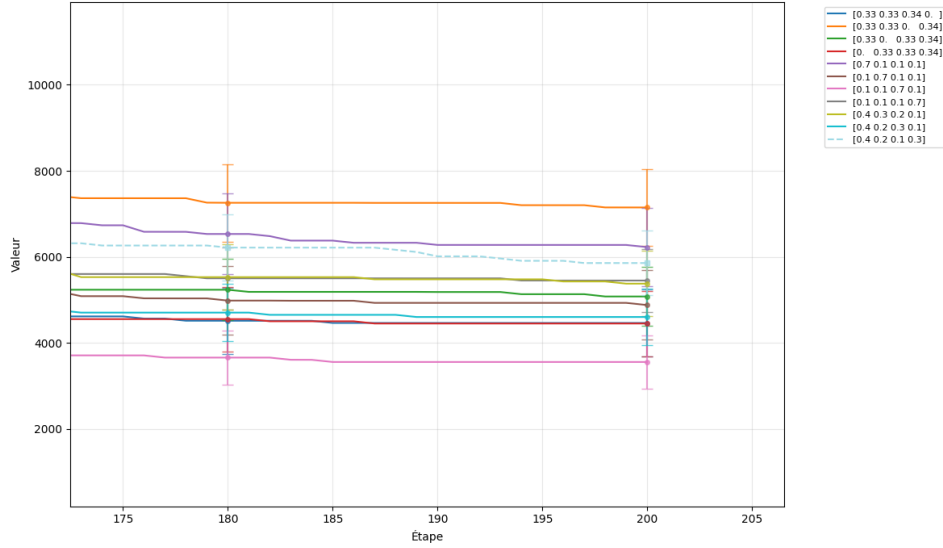
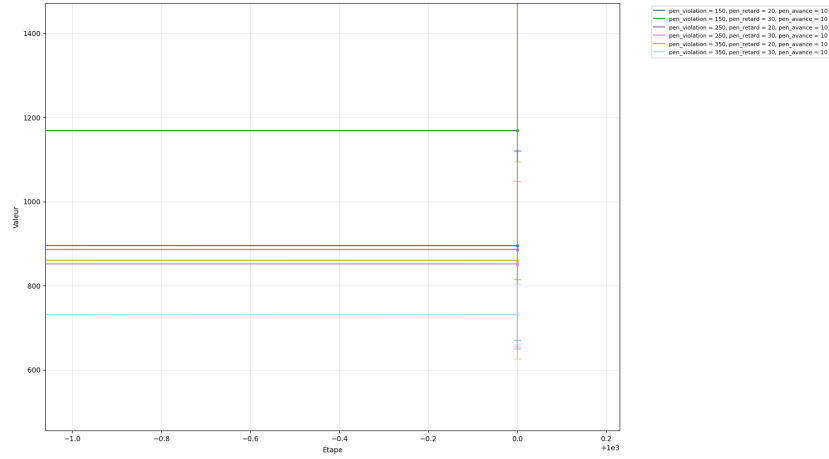

 Figure 6: Comparaison de plusieurs vecteurs à dominante **scramble**.


Figure 7: Exemple de comparaison sur les pénalités.

présentent encore 2 à 3 villes en retard.

- **Instance de concours** (~100 villes) : une solution admissible (sans avance ni retard) est trouvée sans difficulté ; en revanche, le score varie d'un essai à l'autre, ce qui suggère que la solution ainsi obtenue n'est pas systématiquement optimale.

Nous savons depuis la fin du projet que l'instance de concours dispose d'une solution optimale publiquement connue, dont le score est sensiblement meilleur que celui que nous rapportons. Nous indiquons ce fait par souci d'honnêteté ; nous n'avons pas réalisé de comparaison chiffrée a posteriori car notre travail est resté dans le périmètre des essais effectués pendant le projet.

6 Discussion et limites

6.1 Erreur d'implémentation sur la pénalité d'avance

En relisant la fonction de score après le rendu, nous avons identifié une erreur dans le calcul de l'avance. L'extrait en cause est le suivant :

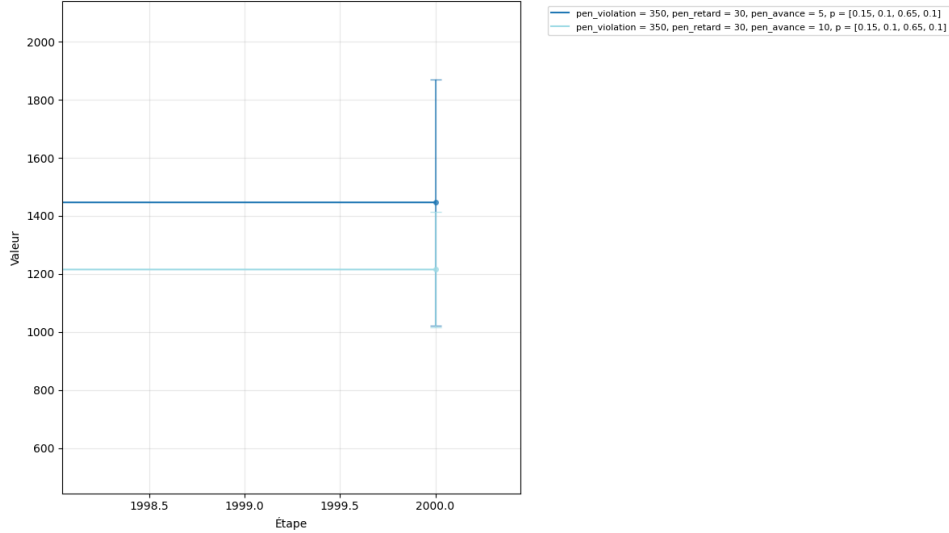


Figure 8: Comparaison des scores finaux pour $p_a = 5$ et $p_a = 10$, autres paramètres égaux.

```
if duree < next_start:
    duree = next_start          # <-- duree écrasée d'abord
    avance += next_start - duree # <-- vaut donc 0
```

Après l'affectation `duree = next_start`, la quantité `next_start - duree` vaut 0. La variable t_{avance} est donc systématiquement nulle, et le terme $p_a \cdot t_{avance}$ ne contribue jamais au score. À paramétrage par ailleurs égal et à graine aléatoire fixée, le recuit produit la *même trajectoire* pour toute valeur de p_a — ce que nous avons vérifié a posteriori sur le code restructuré.

Cette erreur soulève une question : pourquoi la figure relative à p_a fait-elle apparaître un écart entre les groupes $p_a = 5$ et $p_a = 10$, alors que l'algorithme, théoriquement, ne devrait pas distinguer ces deux configurations ?

L'explication la plus plausible tient à la méthodologie de comparaison plutôt qu'à un effet réel de p_a :

- Les deux configurations comparées dans une même session de balayage partagent l'état global de `np.random` : la première configuration consomme une portion de la séquence aléatoire, la deuxième commence là où la première s'est arrêtée. Les deux groupes ne voient donc pas les mêmes graines, et la comparaison n'isole pas l'effet de p_a .
- L'intervalle de confiance affiché borne la moyenne d'une configuration à partir de la variance des essais qu'elle contient. Il ne reflète pas la variabilité induite par le choix de la séquence aléatoire entre configurations. L'écart visible entre les deux moyennes peut donc être plus large que ne le laisse entendre la séparation des intervalles.

Ces deux points combinés sont compatibles avec l'écart observé. Une comparaison plus rigoureuse nécessiterait de fixer une graine par essai, partagée entre configurations, ce qui n'a pas été fait dans le projet et que nous n'avons pas refait depuis. Le code restructuré expose désormais un argument `rng_seed` permettant ce type de protocole.

6.2 Coefficient sur les violations

Le passage de $p_v = 10\,000$ à $p_v = 350$ a clairement amélioré la capacité de l'algorithme à explorer des solutions de meilleure qualité une fois les fenêtres satisfaites. Au-delà de ce constat qualitatif, nous n'avons pas conduit d'étude systématique de l'effet de p_v sur l'instance de concours.

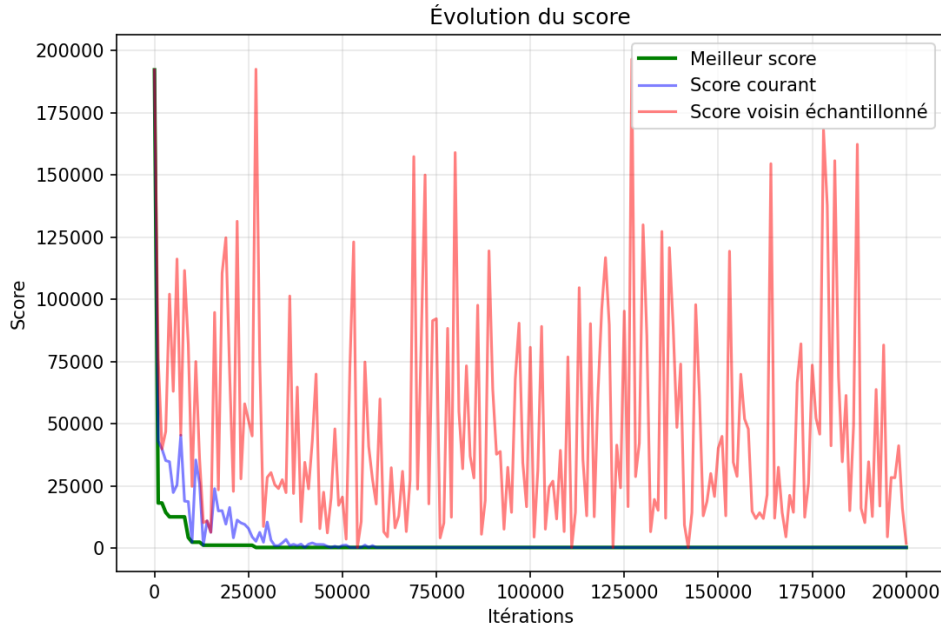


Figure 9: Recuit ayant atteint la solution optimale sur l'instance 1.

6.3 Limites de la fonction de voisinage

Sur l'instance 3, l'algorithme stagne avec 2 à 3 violations résiduelles. Nous pensons qu'une fonction de voisinage informée par l'état courant — par exemple un opérateur qui choisit la position de réinsertion en fonction des fenêtres temporelles plutôt qu'aléatoirement — aurait été nécessaire pour franchir ce palier. Nous n'avons pas eu le temps de la mettre en œuvre.

6.4 Écart à l'optimum sur l'instance de concours

Plusieurs facteurs peuvent expliquer l'écart entre notre meilleur score et l'optimum publiquement connu :

- Une intensification insuffisante en fin de recuit : la température finale et la longueur de la phase de basse température n'ont pas été calibrées spécifiquement pour cette instance.
- Un voisinage non spécialisé pour l'aspect temporel : nos quatre opérateurs travaillent sur la structure de permutation sans tenir compte des fenêtres.
- Le bug d'avance : la pénalité d'avance ne contribuant pas au score, l'algorithme n'avait pas d'incitation à regrouper les visites compatibles, ce qui aurait pu favoriser des tours plus courts dans la phase d'intensification.
- Le choix de la graine et la dispersion entre essais : un nombre limité d'essais sur l'instance de concours a pu masquer des solutions meilleures atteignables avec une autre initialisation.

7 Conclusion

Le projet a permis d'implémenter et de calibrer un recuit simulé pour le TSPTW. Sur les instances 1 et 2, l'algorithme retrouve la solution optimale fournie. Sur l'instance 3 il atteint des solutions admissibles et presque admissible mais les scores finaux restent très élevés, même avec 0 violations. Sur l'instance de concours il produit une solution admissible dont nous ne pouvons pas garantir l'optimalité, et nous savons depuis la fin du projet qu'une meilleure solution existe.

Au-delà des résultats, le projet a mis en évidence des points méthodologiques importants : la nécessité de calibrer la température en cohérence avec le nombre d'itérations, la limite des

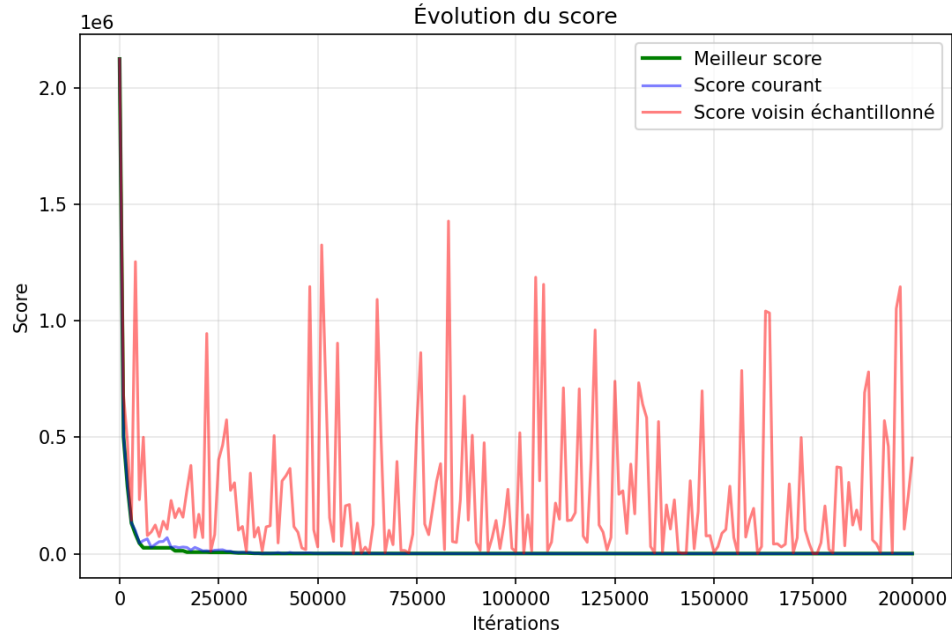


Figure 10: Recuit ayant atteint la solution optimale sur l'instance 2.

comparaisons par moyennage lorsque l'état aléatoire n'est pas contrôlé, et le coût d'une erreur d'implémentation discrète comme celle que nous avons identifiée a posteriori sur la pénalité d'avance. Le code a été restructuré en modules pour permettre des reprises ultérieures, et expose désormais un argument de graine pour faciliter les comparaisons rigoureuses.

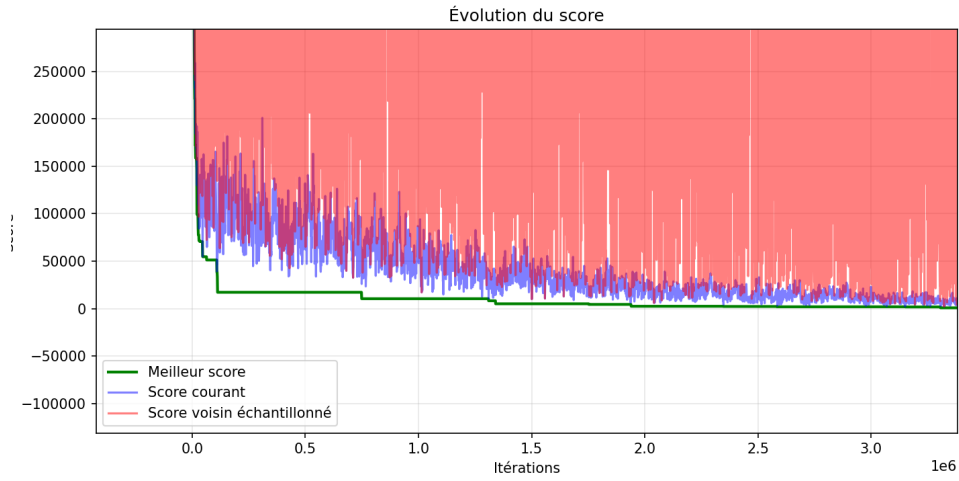


Figure 11: Zoom sur le recuit ayant atteint la solution optimale sur l'instance 3.

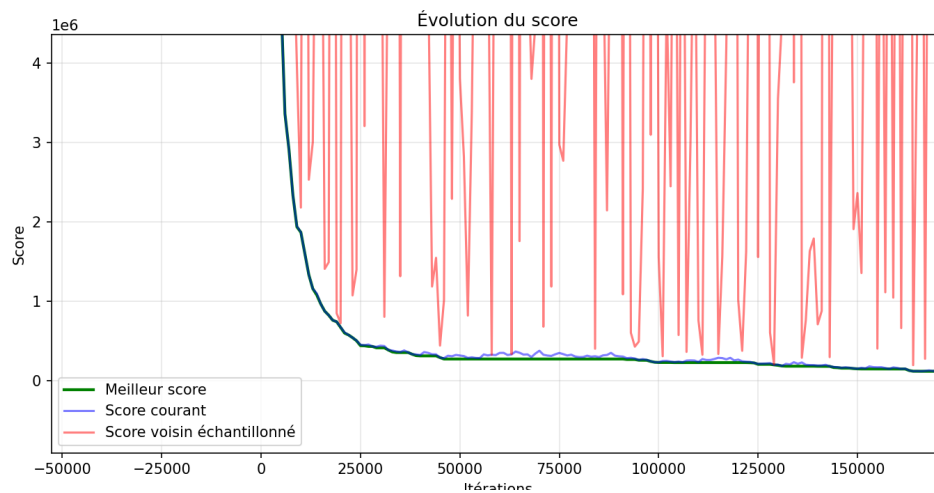


Figure 12: Zoom sur le recuit ayant atteint la solution optimale sur l'instance concours.